

I am working on Task 1 of my Connect Four Vibe Coding lab.

Here is my current board.py code:

```
ROWS, COLS = 6, 7
```

```
class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]
```

Show more ▾



The only necessary change is in `winner()` : add the two diagonal directions to the existing horizontal and vertical directions.

Python

Run

```
ROWS, COLS = 6, 7

class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        if not 0 <= col < COLS:
```

```
        return None
    for r in range(ROWS - 1, -1, -1):
        if self.grid[r][col] == ".":
            self.grid[r][col] = token
            return r
    return None

def full(self):
    return all(self.grid[0][c] != "." for c in range(COLS))

def winner(self, token):

    directions = [
        (0, 1), # horizontal
        (1, 0), # vertical
        (1, 1), # diagonal down-right
        (1, -1) # diagonal down-left
    ]

    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] != token:
                continue

            for dr, dc in directions:
                if all(
                    0 <= r + i * dr < ROWS and
                    0 <= c + i * dc < COLS and
                    self.grid[r + i * dr][c + i * dc] == token
                    for i in range(4)
                ):
                    return True

    return False
```

```
def print(self):
    print("\n " + " ".join(str(i + 1) for i in range(COLS)))
    for row in self.grid:
        print(" " + " ".join(row))
```

What changed

Only this part was modified:

Python

Run

```
directions = [
    (0, 1),
    (1, 0),
    (1, 1),
    (1, -1)
]
```

The original code had:

- (0, 1) → **horizontal**
- (1, 0) → **vertical**

I added:

- (1, 1) → **diagonal down-right** 
- (1, -1) → **diagonal down-left** 

The existing `all(...)` logic already checks four consecutive positions and automatically prevents going outside the 6×7 board. So **no other part of the `Board` class needed to be changed.**



Your current implementation already satisfies most of Task 2.

Requirement check

1. **Handle draws when the board becomes full** —  Already satisfied
`self.board.full()` is checked immediately after a valid move.
2. **Handle full columns correctly** —  Already satisfied
`self.board.drop()` returns `None` for a full column, and your code prints "Column unavailable.".
3. **Handle invalid moves without modifying the board** —  Already satisfied
Invalid input is rejected before `drop()` is called, and `drop()` itself doesn't modify the board when the column is invalid/full.
4. **Winning move immediately ends the game** —  Already satisfied
`winner()` is checked immediately after the move, before `self.turn` changes and before another input is requested.

5. Keep existing structure —  Already satisfied.
6. Only necessary changes — The current code actually needs **no functional changes** for these requirements.
7. No external libraries —  Already satisfied.

So the corrected `game.py` can remain essentially unchanged:

Python

Run

```
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")
        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()

                if raw == "q":
                    return

                try:
                    col = int(raw) - 1
                except ValueError:
                    print("Enter a column number.")
```

```

        continue
    else:
        col = self.ai.choose_column(self.board)

    if col is None or self.board.drop(col, self.turn) is None:
        print("Column unavailable.")
        if self.turn == "O":
            return
        continue

    if self.board.winner(self.turn):
        self.board.print()
        print(self.turn, "wins!")
        return

    if self.board.full():
        self.board.print()
        print("Draw.")
        return

    self.turn = "O" if self.turn == "X" else "X"

```

Important: I did not add unnecessary changes because your existing control flow already handles the Task 2 requirements correctly. The key order is:

valid move → check winner → check draw → switch turn

That order ensures a winning move ends the game immediately, while a full board with no winner produces a draw.



Now implement Task 3 from the README in my existing ai.py.

Current ai.py:

```
import random
```

```
class AI:
```

```
    def choose_column(self, board, me="O", opponent="X"):
```

```
        # Starter AI is deliberately basic; students improve it in Task 3.
```

Show more ▾

Approach

The AI should evaluate moves in this order:

1. Find all **legal columns**.
2. Temporarily place "o" in each legal column and check `board.winner("O")` . If one wins, immediately choose it.
3. Otherwise, temporarily place "x" in each legal column and check `board.winner("X")` . If one wins, choose that column to block.
4. Otherwise, choose a random legal column.
5. Undo every hypothetical move so the real board is never permanently changed.
6. Return `None` if there are no legal columns.

Since `Board.drop()` already handles full columns, we can use it for the hypothetical moves and then restore the affected cell.

Complete `ai.py`

Python

Run

```
import random

class AI:
    def choose_column(self, board, me="O", opponent="X"):

        legal = [c for c in range(7) if board.grid[0][c] == "."]

        if not legal:
            return None

        # Check for an immediate winning move.
        for col in legal:
            row = board.drop(col, me)

            if board.winner(me):
                board.grid[row][col] = "."
                return col

            board.grid[row][col] = "."

        # Check if the opponent has an immediate winning move.
        for col in legal:
            row = board.drop(col, opponent)

            if board.winner(opponent):
                board.grid[row][col] = "."
```

```

        return col

    board.grid[row][col] = "."

    # Otherwise, choose any legal column.
    return random.choice(legal)

```

The important part is that every hypothetical `drop()` is immediately undone with:

Python

Run

```
board.grid[row][col] = "."
```

So the AI can test both winning and blocking moves **without permanently modifying the actual game board**.



Now implement Task 4 from the README in my existing game.py.

Current game.py:

```

from board import Board
from ai import AI

```

```

class Game:
    def __init__(self):
        self.board = Board()

```

Show more ▾

Add the feedback **immediately after the actual** `board.drop()` **succeeds**. This is the best location because:

- It runs exactly once for every successful real move.
- Invalid/full-column moves never reach it.
- AI hypothetical moves happen inside `ai.py`, not `game.py`, so they won't trigger the message.
- It works for both "X" and "O".

I'll store the result of `drop()` in `row`, then print the feedback only when `row` is not `None`.

Corrected `game.py`

Python

Run

```
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")
        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()
```

```
        if raw == "q":
            return

        try:
            col = int(raw) - 1
        except ValueError:
            print("Enter a column number.")
            continue
    else:
        col = self.ai.choose_column(self.board)

    row = self.board.drop(col, self.turn)

    if row is None:
        print("Column unavailable.")
        if self.turn == "O":
            return
        continue

    print(self.turn, "placed in column", col + 1)

    if self.board.winner(self.turn):
        self.board.print()
        print(self.turn, "wins!")
        return

    if self.board.full():
        self.board.print()
        print("Draw.")
        return

    self.turn = "O" if self.turn == "X" else "X"
```

The key change is replacing:

Python

Run

```
if col is None or self.board.drop(col, self.turn) is None:
```

with:

Python

Run

```
row = self.board.drop(col, self.turn)
```

```
if row is None:
```

and then adding:

Python

Run

```
print(self.turn, "placed in column", col + 1)
```

This feedback is therefore tied **only to actual successful moves**.



ChatGPT can make mistakes. Check important info.